

Weak text helps alignment, while lip/audio latents carry the speech signal

Recon model | Auto-AVSR preprocessing | Trump silent-reference demo

1. Why audio-centric?

Direct vision -> text -> TTS makes text accuracy the bottleneck.
Our finding: audio quality stays strong when AVSR text is noisy; lip/audio latents matter more.
The model reconstructs Mimi audio latents from visual, text, and timbre conditions.

0.5818

GT text corr

val1000 final

0.5783

AVSR text corr

drop only 0.0035

-0.35

FM corr

sampling route rejected

2. Data processing

lip_avsr.npy	Auto-AVSR lip crop latent
avsr_enc_lipavsr.npy	AVSR visual encoder state
avsr_text_lipavsr.txt	noisy text, weak semantic aid
SmolLM2 hidden	aligned text-token hidden states
speakertimbres	speaker_emb + Mimi prompt stats
target	Mimi audio latent to reconstruct

Reprocessed clips are stored under data/processed/pretrain/* with AVSR-compatible lip crops and text.
This is a model input decision, not just a cleanup step.

Key point: text is intentionally a weak condition. Even high word-error AVSR text can still preserve intelligible speech when the lip/audio path is reliable.

Training/eval data path: raw clip -> face/lip preprocessing -> Auto-AVSR visual/text -> language hidden states -> Mimi audio latent target.
Silent demo path: silent MP4 plus ref audio/video segment; the first 3.04 s reference is encoded as audio prompt and timbre statistics.
The same preprocessing script is used for raw-video inference so evaluation inputs match the new lip_avsr training representation.

9. End-to-end artifacts

scripts/run_raw_video_avsr_recon_pipeline.py runs preprocessing, AVSR text/latent extraction, recon inference, Mimi decode, and muxing.
scripts/gradio_avsr_gui.py exposes the same flow in a simple GUI.
README documents normal video mode, silent/ref mode, and the current prompt-cropping behavior.
poster/build.sh rebuilds this poster into PPTX, PDF, and PNG preview.

Submission-ready example: silent MP4 + 3 s reference -> voiced MP4. The Trump demo is self-contained and reproducible from checked-in assets.

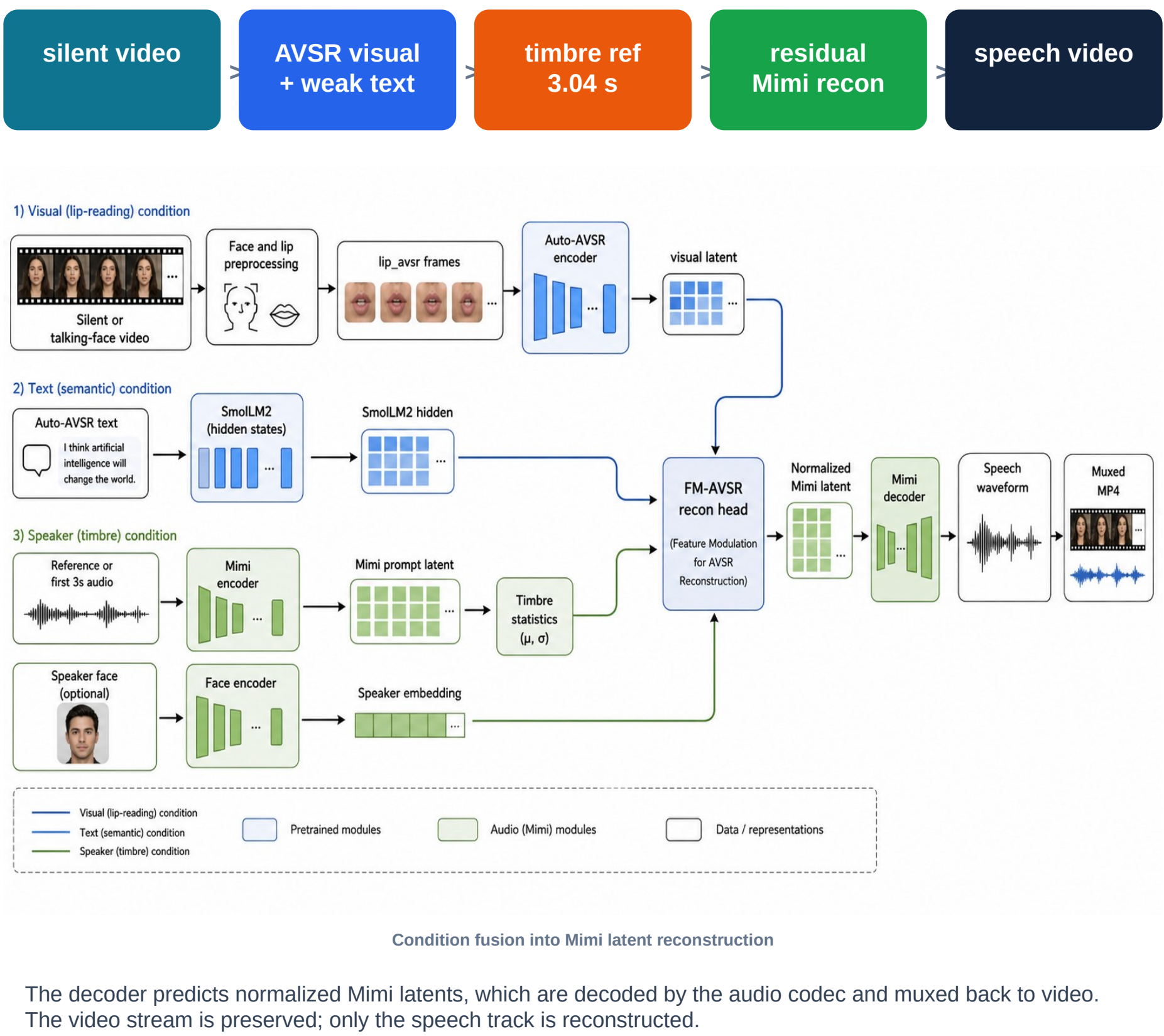
10. Main takeaway

We should not treat lip video as a text-recognition problem followed by speech synthesis.

Lip motion provides phonetic and timing evidence.
Mimi latents preserve acoustic structure and codec-compatible speech detail.
Text tokens provide alignment and coarse semantics, but do not need to be perfect.
Timbre reference improves voice identity, while exposing a prompt-leakage limitation to fix.

Result: a practical raw-video restoration pipeline with stronger audio quality than the FM sampling branch and much lower dependence on exact ASR text.

3. Recon architecture



The decoder predicts normalized Mimi latents, which are decoded by the audio codec and muxed back to video.
The video stream is preserved; only the speech track is reconstructed.

4. Mathematical form

$C = \{\text{visual}, \text{text}, \text{speaker}, \text{audio_prompt}, \text{timbre}\}$

$y_base = f_base(C)$

$\delta = f_residual(C)$

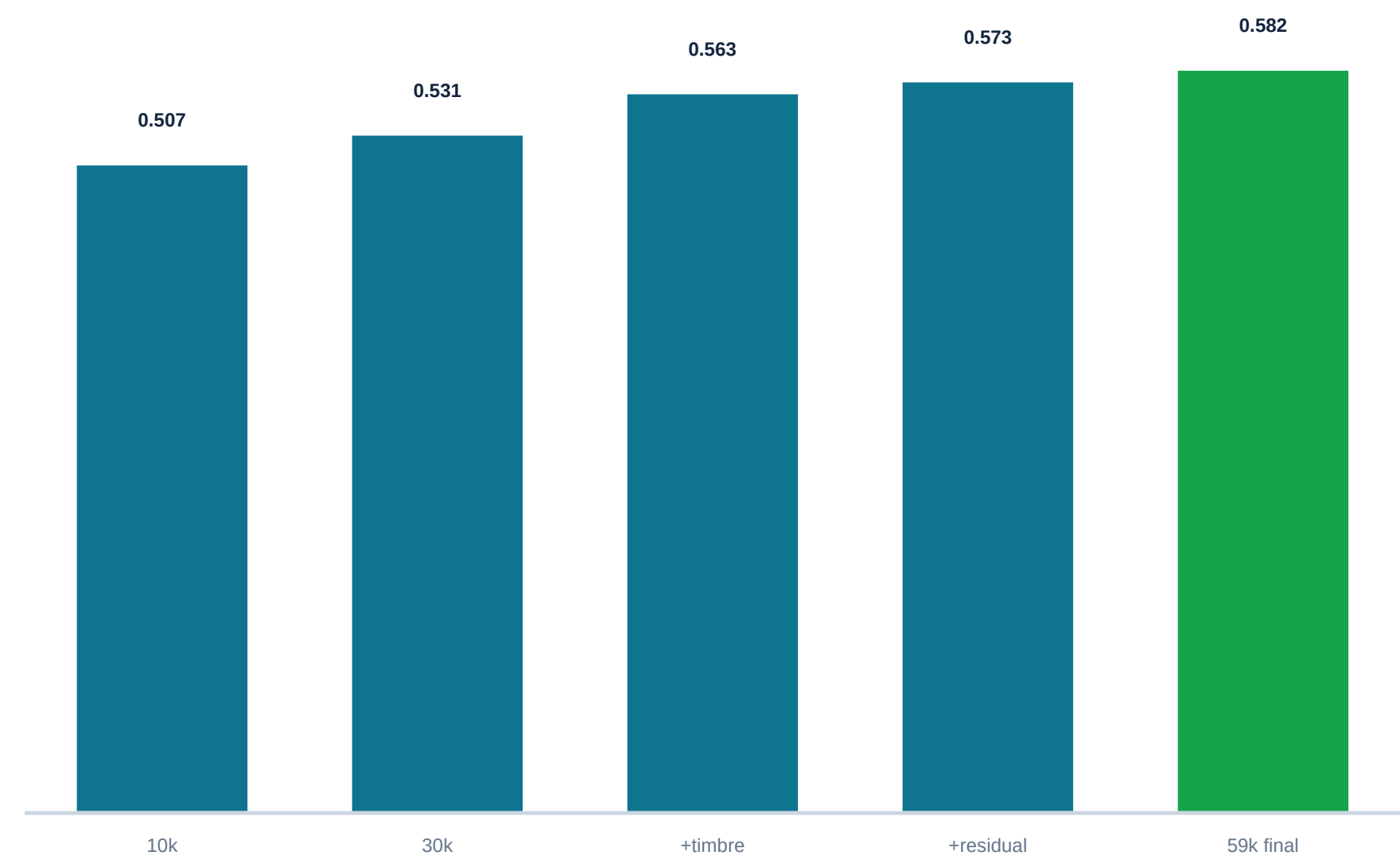
$y_hat = y_base + \delta$

Current inference uses $x_tilde = 0$ and $\tau = 1$. There is no iterative denoise loop; this is a deterministic one-step residual reconstruction of the target Mimi latent.

Why this matters: it avoids the unstable FM sampling route and directly optimizes the latent representation used by the audio codec.

Interpretation: no Gaussian noise state is sampled during inference; τ is kept at 1 during training and evaluation for this endpoint objective.

5. Evaluation trend



Corr improved from the 10k baseline to the final 59k timbre-conditioned model. The AVSR-text run barely drops, supporting the weak-text condition hypothesis.

0.507

10k baseline

small data

0.563

30k + timbre

speaker style

0.582

final 59k

best model

11. Why not cascade?

Cascade: lip -> text -> TTS must solve exact word recovery before generating audio.
Our route: lip/audio latent reconstruction can tolerate noisy text because visual and audio conditions dominate.
Empirical check: replacing GT text with AVSR text changes corr from 0.5818 to 0.5783.

This is the central architectural argument: use text as support, not as the only bridge between video and sound.

6. Trump silent-reference demo

Input: 26.57 s silent Trump video. Reference: final 3.02 s audio segment. Output: 23.55 s generated post-prompt video.



Auto-AVSR text: "OUR COUNTRY IS WINNING ... WE'RE WINNING SO MUCH ... WE WINNING TOO MUCH ..."

The listening export drops the first 3.04 s because the current timbre prompt can be copied at the beginning.

Recommended ref mode: use an unmasked segment from the same video when available.
Zero-ref mode runs, but timbre is weaker and less speaker-specific.
The generated asset is checked in under data/assets/trump_silent_ref_demo/.

Demo phrase: win-win-win / "WE'RE WINNING SO MUCH"

7. What is new?

Residual endpoint reconstruction replaces FM noise sampling.
Timbre/reference conditioning improves speaker style over plain regression.
Text is demoted to an auxiliary alignment signal instead of the primary content bottleneck.
AVSR-compatible data preprocessing makes raw-video inference match training inputs.

The contribution is the full architecture and evidence: data-compatible AVSR latents, weak text conditioning, residual endpoint recon, and a reproducible silent-video demo.

8. Current limitation

The audio_prompt is temporal: (38, 512) Mimi frames from the first 3.04 s. It provides strong timbre control, but the model can learn to copy prompt content. Next step: fixed-size timbre embedding or random prompt dropout/windowing during training.

Current mitigation: crop first 3.04 s from listening exports.
Better direction: speaker-only timbre embedding independent of utterance time.
Training idea: random reference windows, content dropout, or contrastive speaker loss.

12. Files to show

poster/fm_avsr_poster.pptx
poster/fm_avsr_poster.pdf
poster/fm_avsr_poster_preview.png
data/assets/trump_silent_ref_demo/*.mp4
eval_out/trump_raw_prompt_pipeline/.../avsr_text_lipavsr.txt

All paths are relative to the fm-avsr-cleanup worktree.